



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Organización y Arquitectura de Computadoras

Tarea 1. Raíz cuadrada en lenguaje ensamblador

Grupo 1

Ortega de la Paz Rafael 420054085

19 de octubre de 2026

Objetivo

Implementar en lenguaje ensamblador, dentro de MicroPython y sobre la Raspberry Pi Pico 2, el cálculo de la raíz cuadrada de un número de dos maneras: la parte entera y el residuo mediante restas sucesivas de números impares, usando solo registros de propósito general, y el valor en punto flotante mediante la instrucción VSQRT de la unidad de punto flotante.

Introducción

La Raspberry Pi Pico 2 utiliza el microcontrolador RP2350, que integra dos núcleos Arm Cortex-M33 a 150 MHz. A diferencia de la Pico original (RP2040, núcleos Cortex-M0+), cada núcleo Cortex-M33 incluye una unidad de punto flotante (FPU) de precisión simple con 32 registros de 32 bits, S0 a S31, que ejecuta en hardware la suma, la resta, la multiplicación, la división y la raíz cuadrada. La FPU no ofrece funciones trascendentes como seno, coseno o logaritmo; esas deben construirse a partir de las operaciones básicas.

MicroPython permite escribir funciones en ensamblador Thumb dentro de un programa en Python mediante el decorador *@micropython.asm_thumb*. Los primeros cuatro argumentos de la función llegan en los registros r0 a r3 y el valor de regreso se toma de r0. Cuando el argumento es un arreglo (*array*), el registro recibe la dirección de su primer elemento, de modo que el ensamblador puede leer y escribir la memoria del arreglo con instrucciones de carga y almacenamiento (ldr/str para enteros, vldr/vstr para flotantes). Como la memoria se direcciona por bytes y cada dato de 32 bits ocupa cuatro, el desplazamiento del elemento i es $4 \cdot i$.

El método de restas sucesivas se basa en que la suma de los primeros n números impares es igual a n^2:

$$1 + 3 + 5 + \ldots + (2n - 1) = n^2$$

Por lo tanto, si a un número N se le restan los impares 1, 3, 5, … mientras el resultado no se vuelva negativo, la cantidad de restas válidas es la parte entera de su raíz cuadrada y lo que sobra es el residuo, de forma que N = raíz² + residuo. El método solo necesita comparaciones, restas y sumas, por lo que funciona en cualquier procesador, incluso sin FPU. En cambio, en la Pico 2 la FPU calcula la raíz en punto flotante con una sola instrucción.

Desarrollo

Método A: restas sucesivas de impares

La rutina *raiz_entera* recibe en r0 la dirección de un arreglo de enteros de 32 bits con tres posiciones: el número *N*, la raíz y el residuo. La Tabla 1 resume el uso de los registros.

Tabla 1. Registros utilizados en el método de restas.

Registro	Contenido
r0	Dirección del arreglo [N, raíz, residuo]
r1	N, que se reduce en cada resta
r2	Raíz: contador de restas válidas
r3	Impar actual: 1, 3, 5, …

El ciclo compara r1 con r3 mediante *cmp*; si *N* ya es menor que el impar siguiente, la instrucción de salto condicional *blt* termina el ciclo. En caso contrario se resta el impar (*sub*), se incrementa la raíz (*add* r2, r2, 1) y se pasa al siguiente impar (*add* r3, r3, 2). Al salir, *str* guarda la raíz y el residuo en los desplazamientos 4 y 8 del arreglo. La Tabla 2 muestra los dos ejemplos vistos en clase.

Tabla 2. Ejecución del algoritmo para 25 y 27.

Paso	√25: valor	√27: valor
Inicio	25	27
− 1	24	26
− 3	21	23
− 5	16	18
− 7	9	11
− 9	0	2
Siguiente impar: 11	No se resta (0 < 11)	No se resta (2 < 11)
Resultado	raíz = 5, residuo = 0	raíz = 5, residuo = 2

Método B: instrucción VSQRT de la FPU

La rutina *raiz_fpu* recibe en r0 la dirección del dato de entrada y en r1 la del resultado, ambos en arreglos de flotantes de 32 bits. Carga el valor en s0 con *vldr*, calcula la raíz con *vsqrt*(s2, s0) y guarda s2 en memoria con *vstr*. La función de Python que la llama rechaza los números negativos antes de enviarlos a la FPU, porque su raíz no es un número real.

Pruebas y resultados

El programa se ejecutó en la Pico 2 desde el entorno Thonny. Para el método A se probaron los casos límite 0, 1 y 2, los ejemplos de clase 25 y 27, y valores mayores hasta 65 535. En cada caso el programa verifica que raíz² + residuo = *N* y que la raíz coincide con la de la biblioteca *math*. Para el método B se compara el resultado de VSQRT contra *math.sqrt*. Al final, un modo interactivo permite introducir cualquier número. La Figura 1 muestra la salida.

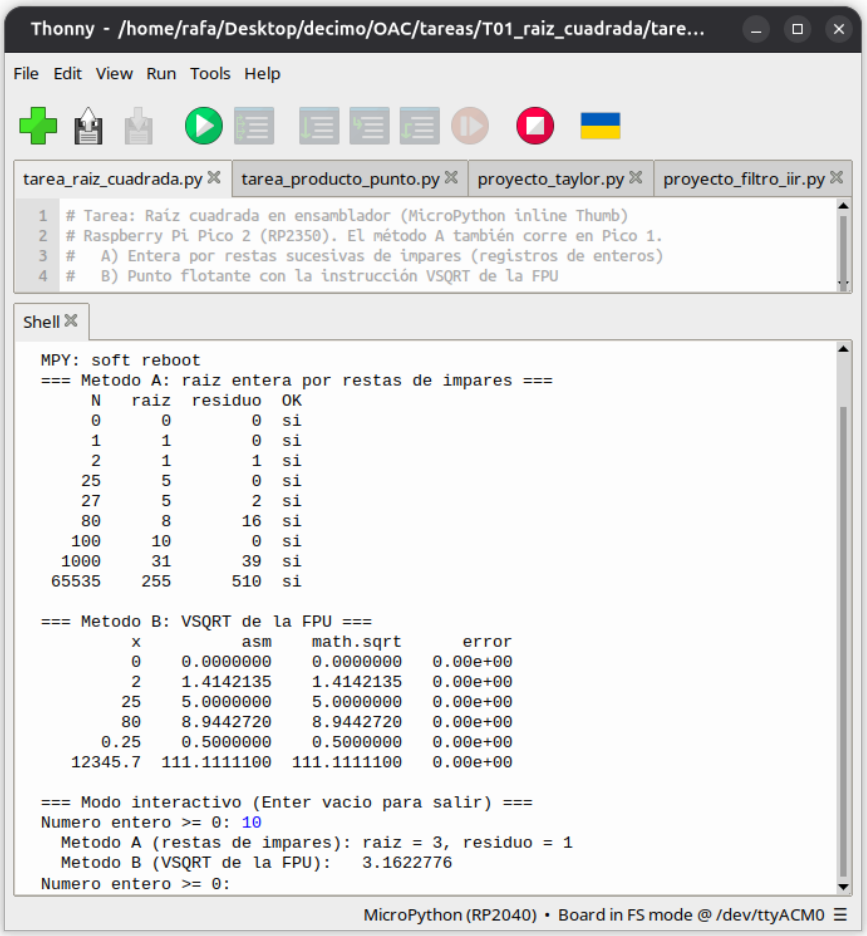


Figura 1. Salida del programa en la Pico 2: método A, método B y modo interactivo con N = 10.

Todos los casos del método A pasaron la verificación. Por ejemplo, 80 da raíz 8 y residuo 16 (64 + 16 = 80), y 65 535 da raíz 255 y residuo 510. En el modo interactivo, para 10 se hicieron tres restas (10 − 1 − 3 − 5 = 1), así que la raíz entera es 3 con residuo 1, mientras que VSQRT entrega 3.1622776. El método B coincidió con *math.sqrt* en todos los casos, con diferencia cero.

La diferencia entre ambos métodos está en el costo: el método A repite el ciclo tantas veces como vale la raíz (255 iteraciones para 65 535), mientras que el método B resuelve cualquier valor con una sola instrucción de hardware y además entrega los decimales.

Conclusiones

Se implementó la raíz cuadrada en ensamblador de dos formas y ambas dieron resultados correctos en la Pico 2. El método de restas de impares demuestra que una operación compleja puede construirse solo con comparaciones, restas y saltos condicionales sobre registros enteros, por lo que también funcionaría en la Pico 1. La instrucción VSQRT muestra la ventaja de contar con una FPU: el mismo cálculo, con decimales, se reduce a una instrucción. La práctica también sirvió para manejar el paso de arreglos por dirección y el uso de desplazamientos de 4 bytes para leer y escribir datos de 32 bits.

Bibliografía

Arm Limited. (2023). *Armv8-M architecture reference manual* (DDI0553). Arm Limited.

Arredondo Garza, J. A. de J. (2026). *Organización y Arquitectura de Computadoras: clases del semestre 2027-1* [Exposiciones en clase]. Facultad de Ingeniería, Universidad Nacional Autónoma de México.

MicroPython. (s. f.). *Inline assembler for Thumb2 architectures*. Recuperado el 19 de octubre de 2026 de https://docs.micropython.org/en/latest/reference/asm_thumb2_index.html

Raspberry Pi Ltd. (2025). *RP2350 datasheet: A microcontroller by Raspberry Pi*. https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf

Warren, H. S. (2012). *Hacker's delight* (2.ª ed.). Addison-Wesley.

Anexo: código fuente (tarea_raiz_cuadrada.py)

```
# Tarea: Raíz cuadrada en ensamblador (MicroPython inline Thumb)
# Raspberry Pi Pico 2 (RP2350). El método A también corre en Pico 1.
#   A) Entera por restas sucesivas de impares (registros de enteros)
#   B) Punto flotante con la instrucción VSQRT de la FPU
from array import array
import math

# ---------- Método A: restas de impares 1, 3, 5, 7, ... ----------
# r0 -> array('i', [N, raiz, residuo])
@micropython.asm_thumb
def raiz_entera(r0):
    ldr(r1, [r0, 0])      # r1 = N (se va reduciendo)
    mov(r2, 0)            # r2 = raiz (contador de restas)
    mov(r3, 1)            # r3 = impar actual
    label(CICLO)
    cmp(r1, r3)
    blt(FIN)              # si N < impar, termina
    sub(r1, r1, r3)       # N = N - impar
    add(r2, r2, 1)        # raiz += 1
    add(r3, r3, 2)        # siguiente impar
    b(CICLO)
    label(FIN)
    str(r2, [r0, 4])      # guarda raiz
    str(r1, [r0, 8])      # guarda residuo
    mov(r0, r2)           # también la regresa

# ---------- Método B: FPU, VSQRT ----------
# r0 -> array('f', [x]),  r1 -> array('f', [resultado])
@micropython.asm_thumb
def raiz_fpu(r0, r1):
    vldr(s0, [r0, 0])     # s0 = x
    vsqrt(s2, s0)         # s2 = sqrt(x)
    vstr(s2, [r1, 0])     # resultado

def sqrt_fpu(x):
    if x < 0:
        raise ValueError("No se admiten negativos en la FPU")
    entrada = array('f', [x])
    salida = array('f', [0.0])
    raiz_fpu(entrada, salida)
    return salida[0]

# ---------- Pruebas ----------
print("=== Metodo A: raiz entera por restas de impares ===")
print("{:>6} {:>6} {:>8}  {}".format("N", "raiz", "residuo", "OK"))
for n in (0, 1, 2, 25, 27, 80, 100, 1000, 65535):
    d = array('i', [n, 0, 0])
    raiz_entera(d)
    ok = d[1] * d[1] + d[2] == n and d[1] == int(math.sqrt(n))
    print("{:>6} {:>6} {:>8}  {}".format(n, d[1], d[2], "si" if ok else "NO"))

print()
```

```
print("=== Metodo B: VSQRT de la FPU ===")
print("{:>10} {:>12} {:>12} {:>10}".format("x", "asm", "math.sqrt", "error"))
for x in (0.0, 2.0, 25.0, 80.0, 0.25, 12345.678):
    r = sqrt_fpu(x)
    print("{:>10} {:>12.7f} {:>12.7f} {:>10.2e}".format(x, r, math.sqrt(x), abs(r -
math.sqrt(x))))

# ---------- Modo interactivo ----------
print()
print("=== Modo interactivo (Enter vacio para salir) ===")
while True:
    t = input("Numero entero >= 0: ").strip()
    if not t:
        break
    try:
        n = int(t)
        if n < 0:
            raise ValueError
    except ValueError:
        print("  Escribe un entero no negativo")
        continue
    d = array('i', [n, 0, 0])
    raiz_entera(d)
    print("  Metodo A (restas de impares): raiz = {}, residuo = {}".format(d[1], d[2]))
    print("  Metodo B (VSQRT de la FPU):   {:.7f}".format(sqrt_fpu(float(n))))
```